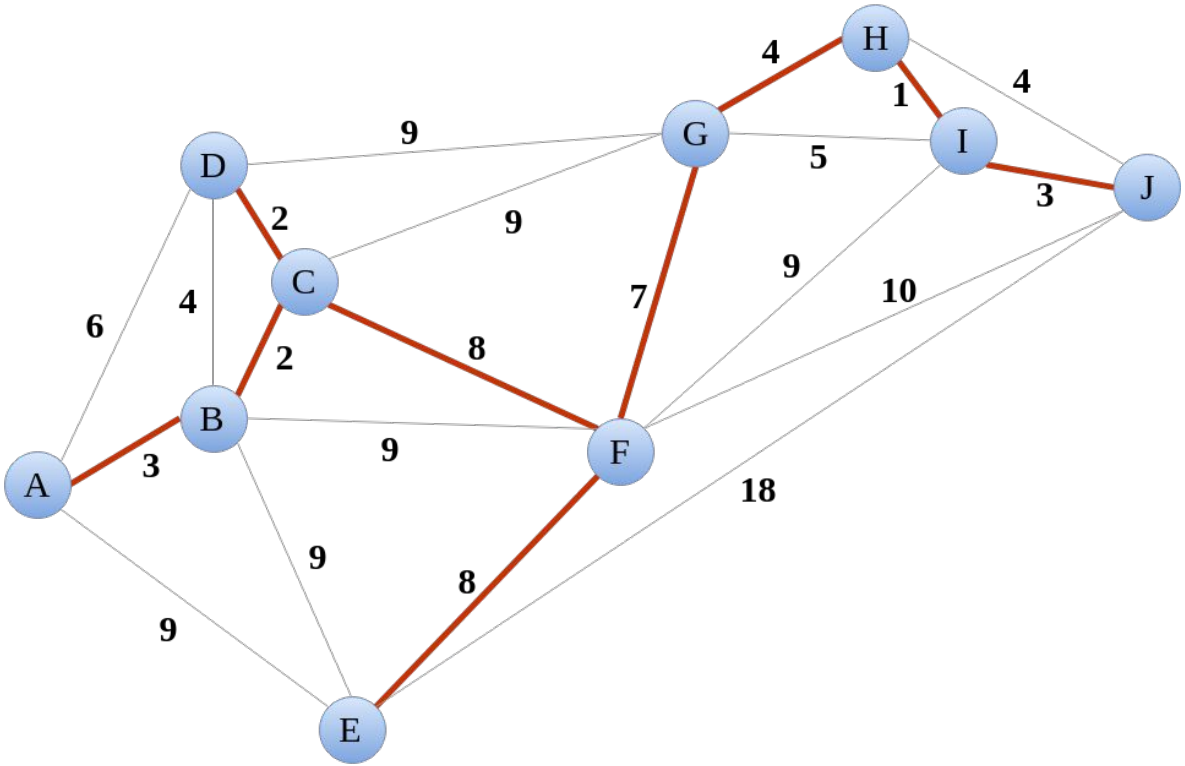
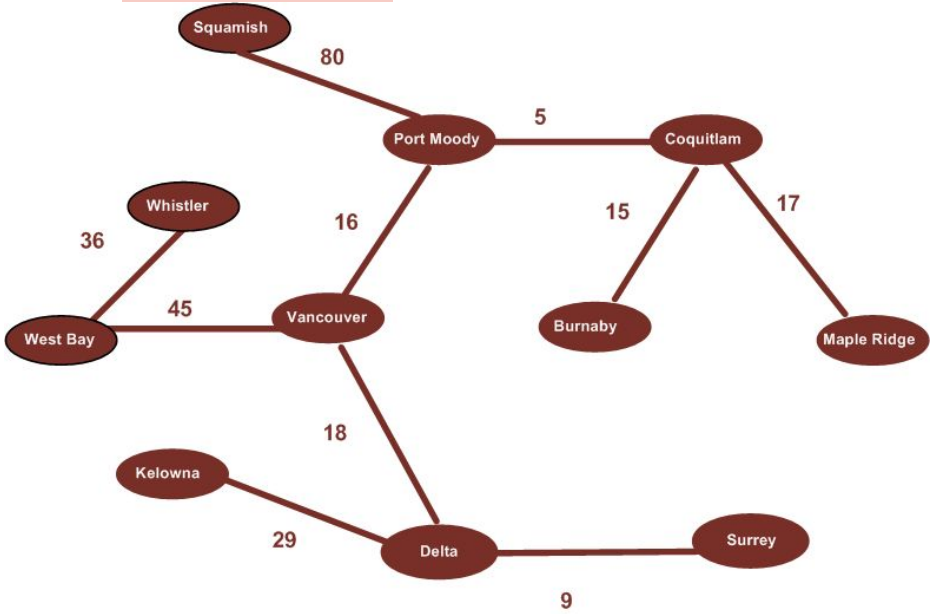
Eco-Friendly Travel Planner

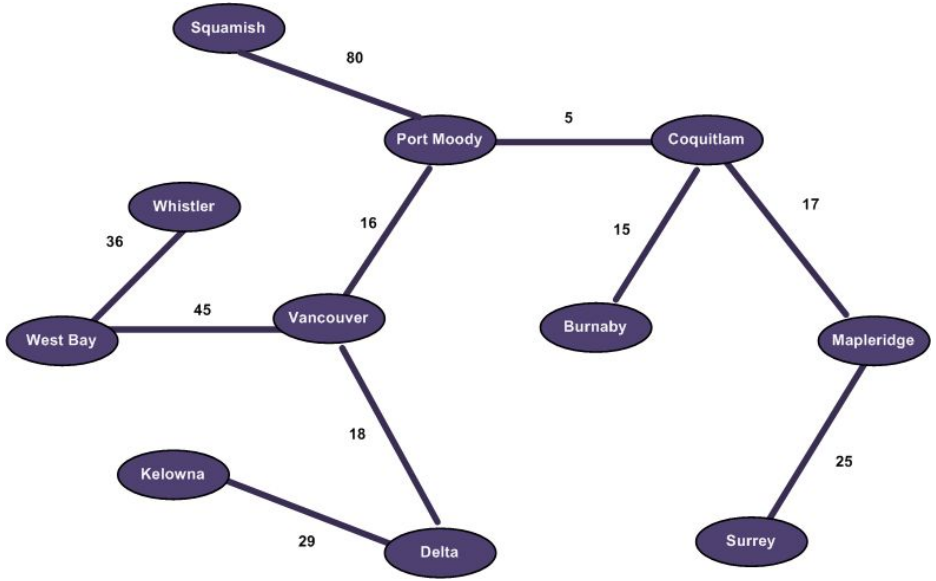
Ankita Kunadia
Jeicy El Hajj
Orkhan Dadashov
Mahsa Khakpour

Minimum Spanning Tree



Kruskal's Algorithm & Prim's Algorithm

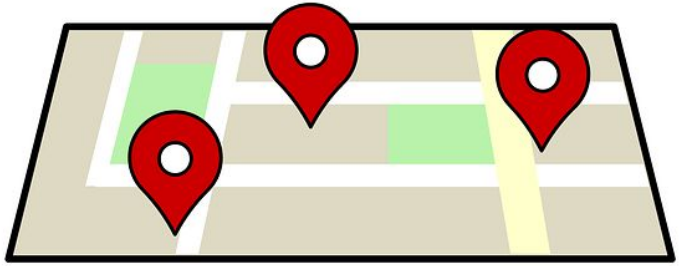




Eco-Friendly Travel Planner

If a traveller is looking to plan a particular multi-city trip while factoring in energy efficiency, in which order should the traveller choose the cities he visits, ensuring minimal fuel usage and thereby being as eco-friendly as possible?

MST algorithms: Kruskal's vs Prim's

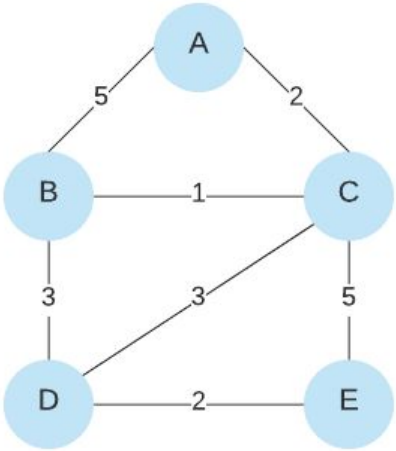


Kruskal's vs Prim's

1. Sort all the edges from lowest to highest weight
2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge
3. Keep repeating step 2 until we reach all vertices

1. Initialize the minimum spanning tree with a random vertex
2. Find the minimum weight edge that connects the tree to new vertices without creating a cycle
3. Keep repeating step 2 until we reach all vertices

Kruskal's Possible Different Outcomes

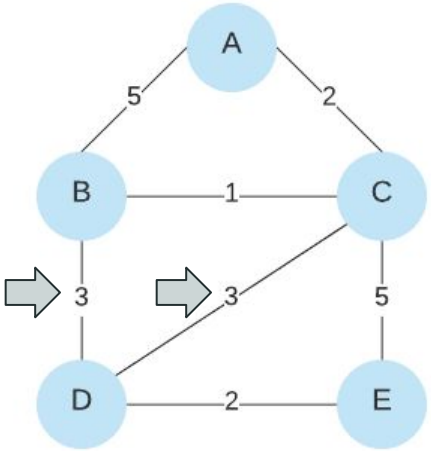


To accomplish step 1 of Kruskal's which is sorting the edges by weight, we notice that there is room for different looking lists for the same graph

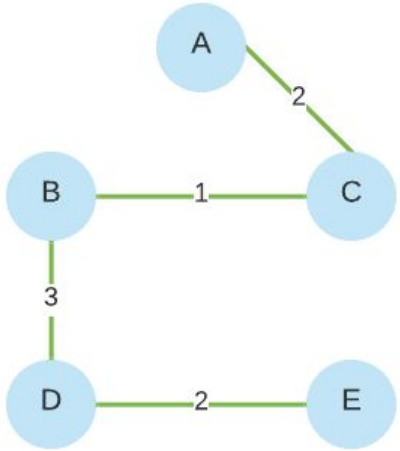
Sorted edges by weight:

BC: 1	BC: 1
AC: 2	DE: 2
DE: 2	AC: 2
BD: 3	DC: 3
DC: 3	BD: 3
AB: 5	CE: 5
CE: 5	AB: 5

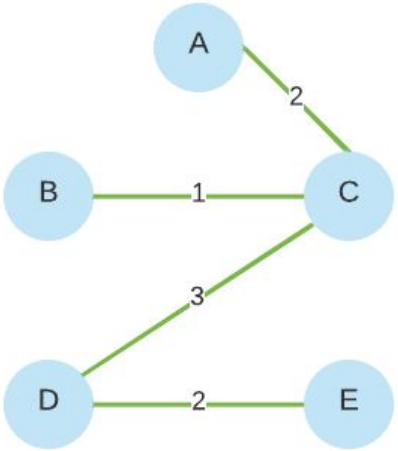
Kruskal's Possible Different Outcomes



BC: 1
AC: 2
DE: 2
BD: 3
DC: 3
AB: 5
CE: 5

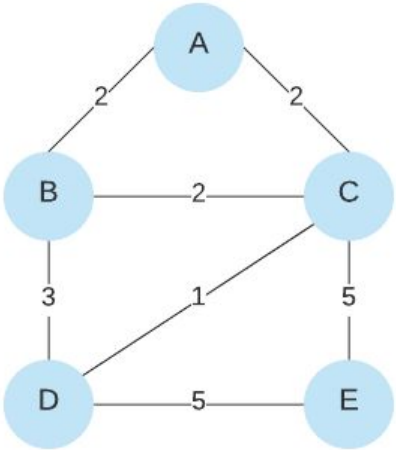


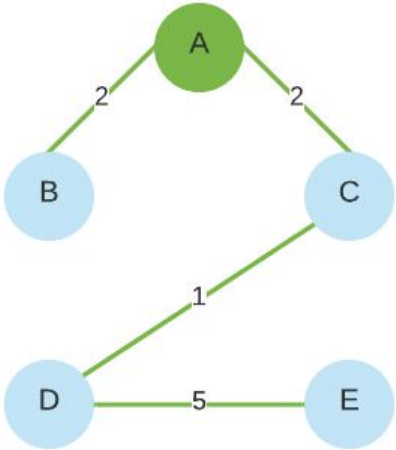
BC: 1
DE: 2
AC: 2
DC: 3
BD: 3
CE: 5
AB: 5

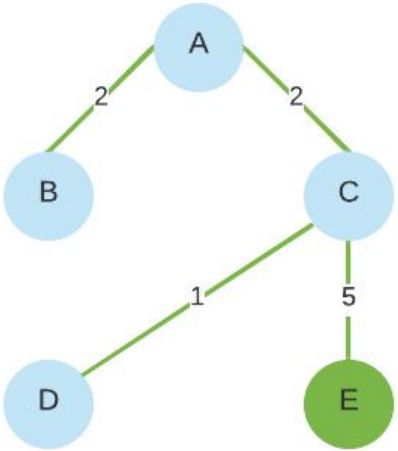


Prim’s Possible Different Outcomes

To initialize the MST graph in prim’s we can choose any arbitrary node as our starting node. This leaves room for different looking trees.



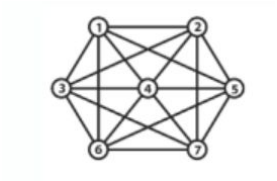




Eco-Friendly Travel Planner MST

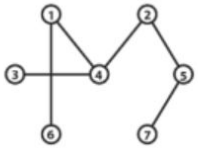
Both of the mentioned algorithms use a greedy approach for finding a MST. But for our Eco-Friendly travel planner, we focused on Kruskal's algorithm for the main advantage that it runs faster with sparsely connected graphs, which is what our travel graphs look like. Also it allows us to work on disconnected components, which could be a case in travel planning.

Kruskal’s vs. Prim’s



Prim’s

Prim's algorithm requires the graph to be connected. It starts finding solutions from a random vertex and by adding the next cheapest vertex. It is efficient to use Prim's algorithm in dense graphs (the number of edges is close to the maximal number of edges), for the case when the number of edges is high $E=O(V^2)$ → number of edges is almost equal to (V^2).



Kruskal’s

Kruskal algorithm works using disconnected graphs.It starts finding a solution for the problem from the cheapest edge by adding the next cheapest edge to the solution. It is efficient to use Kruskal’s algorithm for sparse graphs (there are not many connections among vertices).

Time & Space Complexity

Kruskal's runs in $O(E \log V) = O(V^2 \log V)$, while Prim's runs in $O(E + V \log V) = O(V^2 + V \log V) \Longrightarrow O(V^2)$. So if $E \sim V^2$ (the graph is dense), then we should use Prim's algorithm as it has better runtime complexity (with Fibonacci Heap implementation).

Space Complexity of Kruskal's algorithm is : $O(|E| + |V|)$, since Disjoint Set Data Structure takes $O(|V|)$ space to keep track of the roots of all the vertices and another $O(|E|)$ space to store all edges in a sorted order.

Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
A = ∅
for each vertex v ∈ G.V
    MAKE-SET(v)
sort the edges of G.E into nondecreasing order by weight w
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
return A
```

We call MAKE-SET function |V| time but run time of Make-SET is O(1)

```
MST-KRUSKAL(G, w)
A = ∅
for each vertex v ∈ G.V
    MAKE-SET(v)
sort the edges of G.E into nondecreasing order by weight w
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
return A
```

Sorting can be done in |E| * log (|E|) run time

Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
A = ∅
for each vertex v ∈ G.V
    MAKE-SET(v)
sort the edges of G.E into nondecreasing order by weight w
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
return A
```

We will call O(|E|) times FIND-SET
Run time for this call will be O(1)

```
MST-KRUSKAL(G, w)
A = ∅
for each vertex v ∈ G.V
    MAKE-SET(v)
sort the edges of G.E into nondecreasing order by weight w
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
return A
```

Union will be called O(V) times,
and runtime of each UNION is log(V)

Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
A = ∅
for each vertex v ∈ G.V
    MAKE-SET(v)
sort the edges of G.E into nondecreasing order by weight w
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
return A
```

Total run time for this for loop will be O(V log(V))

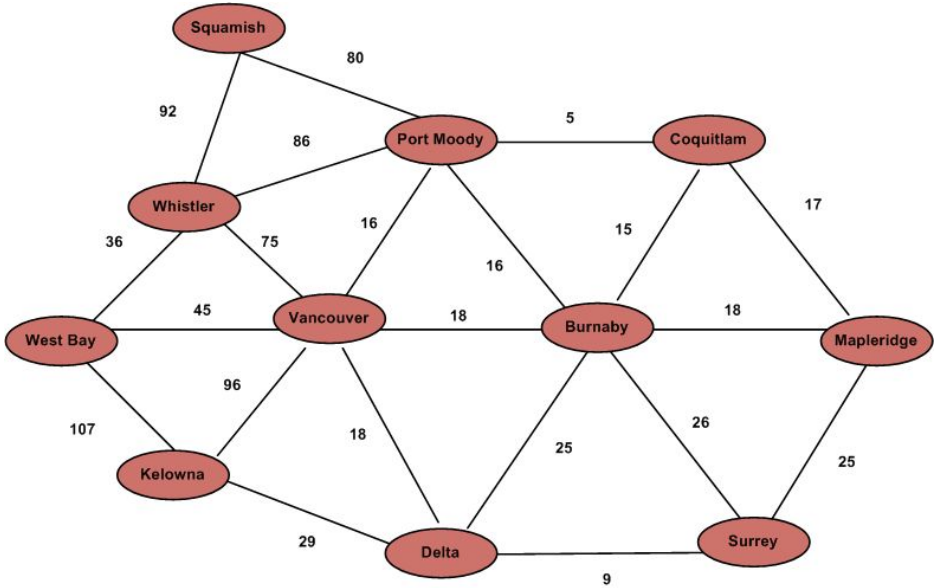
```
MST-KRUSKAL(G, w)
A = ∅
for each vertex v ∈ G.V
    MAKE-SET(v)
sort the edges of G.E into nondecreasing order by weight w
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
return A
```

Total run time for the Kruskal`s algorithm is: O(E log E) = O(E log V).

Eco-Friendly Travel Planner: Kruskal's Algorithm

In this example, let's consider a person who wants to visit 11 cities in BC, Canada, using less gasoline, so he/she needs to go via the shortest path to save gasoline.

The distance is in miles:



Eco-Friendly Travel Planner: Kruskal's Algorithm

First of all, sort the edges.
Second, pick the smallest edge.
Look for the smallest edge that doesn't create a cycle.

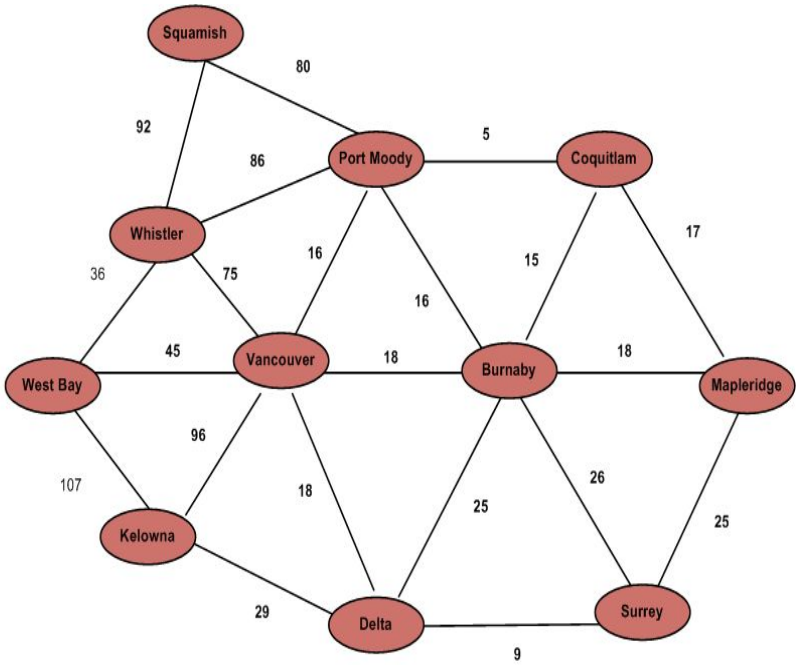
Port Moody	to	Coquitlam	= 5
Delta	to	Surrey	= 9
Coquitlam	to	Burnaby	= 15
Vancouver	to	Port Moody	= 16
Port Moody	to	Burnaby	=16
Coquitlam	to	Maple Ridge	=17
Vancouver	to	Delta	=18
Vancouver	to	Burnaby	=18
Burnaby	to	Maple Ridge	=18
Burnaby	to	Delta	=25
Surrey	to	Maple Ridge	=25
Burnaby	to	Surrey	=26
Kelowna	to	Delta	=29
West Bay	to	Whistler	=36
Vancouver	to	West Bay	=45
Whistler	to	Vancouver	=75
Squamish	to	Port Moody	=80
Whistler	to	Port Moody	=86
Squamish	to	Whistler	=92
Vancouver	to	Kelowna	=96
West Bay	to	Kelowna	=107

Eco-Friendly Travel Planner: Kruskal's Algorithm

The shortest path is the distance between Port Moody and Coquitlam, so we consider it as a possible way, then Delta to Surrey and Coquitlam to Burnaby. Then we have 2 paths with a distance of 16, so we are supposed to choose either Port Moody to Burnaby, or Port Moody to Vancouver, but choosing Port Moody to Burnaby will make a circle, since Kruskal is a MST algorithm, we ignore it.
After choosing Coquitlam to Maple ridge and Vancouver to Delta, will recognize that Vancouver to Burnaby will make a cycle. Same as Burnaby to Maple ridge, Burnaby to Delta, Surrey to Maple ridge and Burnaby to Surrey.
Next we go over Kelowna to Delta, West Bay to Whistler, and Vancouver to West Bay. Again it will be recognized that Whistler to Vancouver will create a cycle, so we will ignore it. The final path is Squamish to Port Moody and we ignore Whistler to Port Moody, Squamish to Whistler, Vancouver to Kelowna and West Bay to Kelowna.

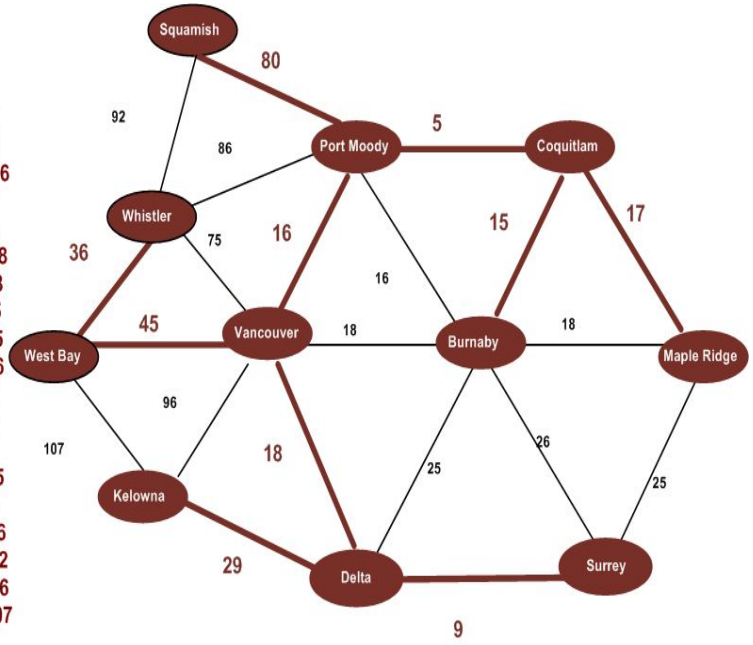
Eco-Friendly Travel Planner: Kruskal's Algorithm

Port Moody	to	Coquitlam	= 5
Delta	to	Surrey	= 9
Coquitlam	to	Burnaby	= 15
Vancouver	to	Port Moody	= 16
Port Moody	to	Burnaby	=16
Coquitlam	to	Maple ridge	=17
Vancouver	to	Delta	=18
Vancouver	to	Burnaby	=18
Burnaby	to	Maple ridge	=18
Burnaby	to	Delta	=25
Surrey	to	Maple Ridge	=25
Burnaby	to	Surrey	=26
Kelowna	to	Delta	=29
West Bay	to	Whistler	=36
Vancouver	to	West Bay	=45
Whistler	to	Vancouver	=75
Squamish	to	Port Moody	=80
Whistler	to	Port Moody	=86
Squamish	to	Whistler	=92
Vancouver	to	Kelowna	=96
West Bay	to	Kelowna	=107

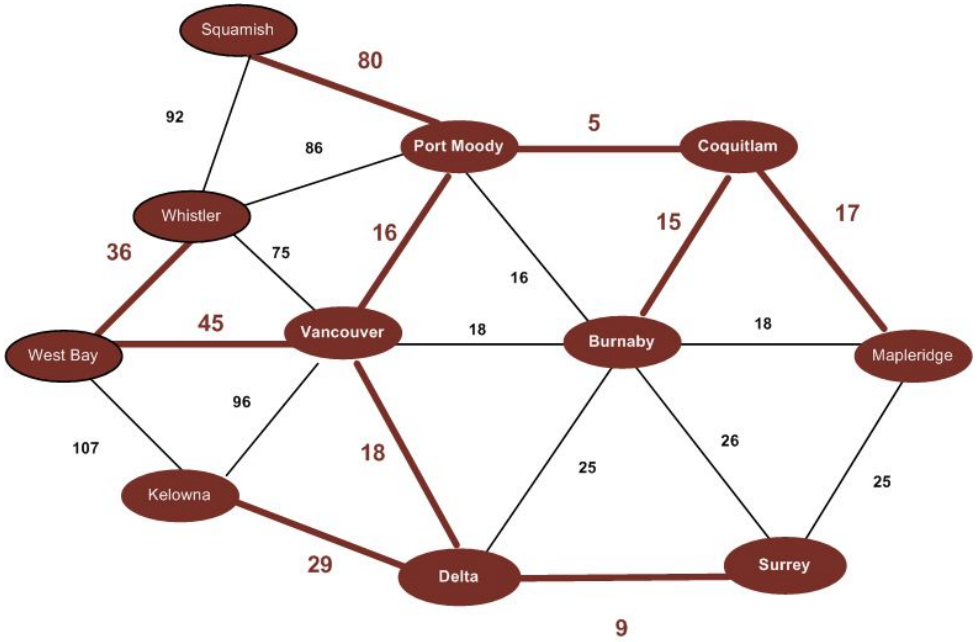


Eco-Friendly Travel Planner: Kruskal's Algorithm

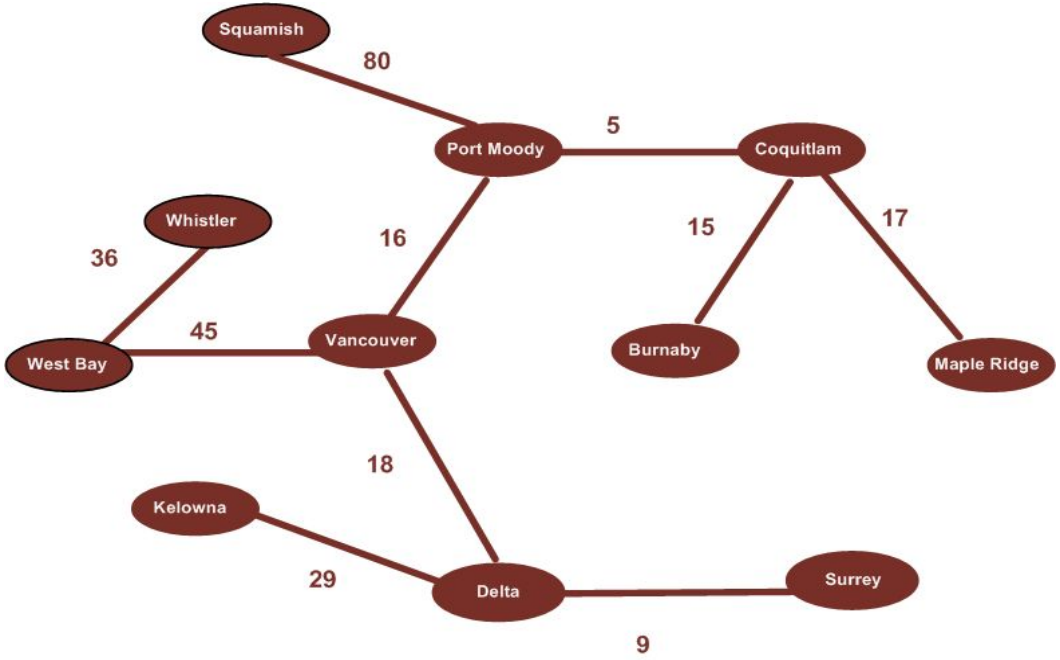
Port Moody	to	Coquitlam	= 5
Delta	to	Surrey	= 9
Coquitlam	to	Burnaby	= 15
Vancouver	to	Port Moody	= 16
Port Moody	**to**	**Burnaby**	**=16**
Coquitlam	to	Maple Ridge	=17
Vancouver	to	Delta	=18
Vancouver	**to**	**Burnaby**	**=18**
Burnaby	**to**	**Maple Ridge**	**=18**
Burnaby	**to**	**Delta**	**=25**
Surrey	**to**	**Maple Ridge**	**=25**
Burnaby	**to**	**Surrey**	**=26**
Kelowna	to	Delta	=29
West Bay	to	Whistler	=36
Vancouver	to	West Bay	=45
Whistler	**to**	**Vancouver**	**=75**
Squamish	to	Port Moody	=80
Whistler	**to**	**Port Moody**	**=86**
Squamish	**to**	**Whistler**	**=92**
Vancouver	**to**	**Kelowna**	**=96**
West Bay	**to**	**Kelowna**	**=107**



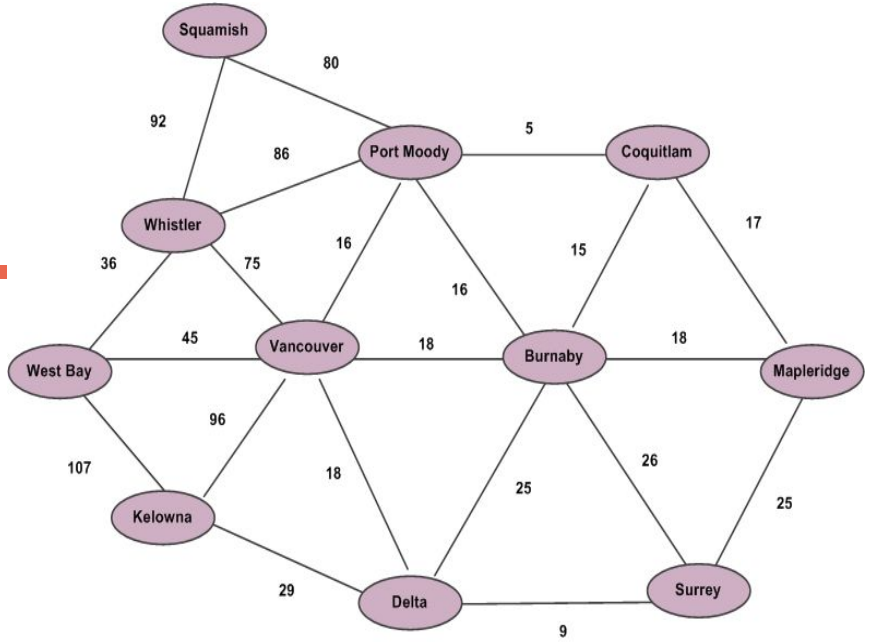
Eco-Friendly Travel Planner: Kruskal's Algorithm



Eco-Friendly Travel Planner: Kruskal's Algorithm



Eco-Friendly Travel Planner: Prim's Algorithm



Eco-Friendly Travel Planner: Prim's Algorithm

First we start creating an empty list, called visited. We will use this list, keep tracking nodes that we have touched.

The second step is picking an arbitrary city to start the algorithm from that node. Let's start with Whistler in our map and add it to the visited list.

Next try all paths reaching from Whistler.

Prim's is a greedy algorithm, so we will choose the shortest path and connect it to the unvisited cities; in this case Whistler to West Bay, so West Bay will be added to the visited list.

Now we will look at all nodes reachable from Whistler and West Bay. The least weighted path is from West Bay to Vancouver, we will go through it and add Vancouver to our visited list.

The next least weighted path is from Vancouver to Port Moody, so we add port Moody to the visited list. The next shortest path is from Port Moody to Coquitlam and we add Coquitlam to the visited list. Same as for Coquitlam to Burnaby, and Coquitlam to Maple Ridge.

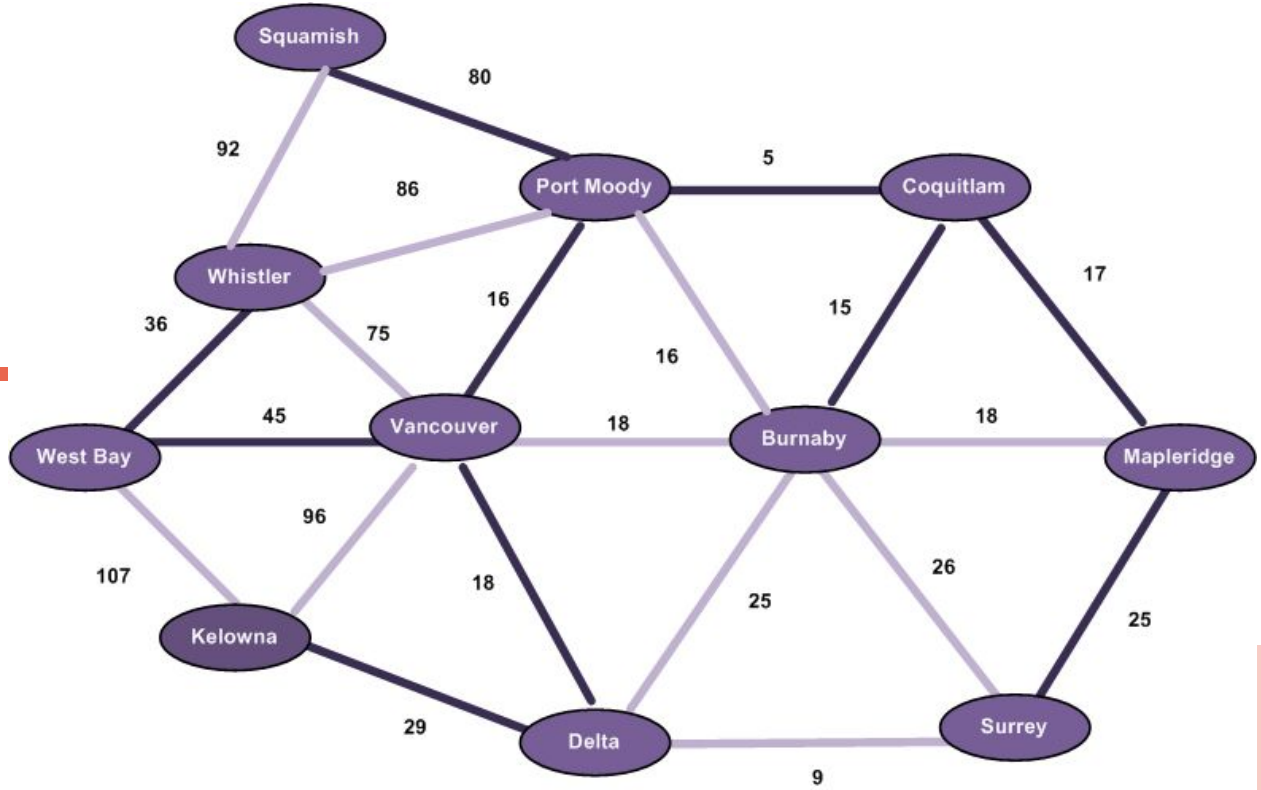
We notice that we can not go through any other part from Burnaby, because all of them are breaking Minimum Spanning Tree rule.

The next path is Vancouver to Delta, so we add Delta to our visited list, then Maple Ridge to Surrey and Delta to Kelowna.

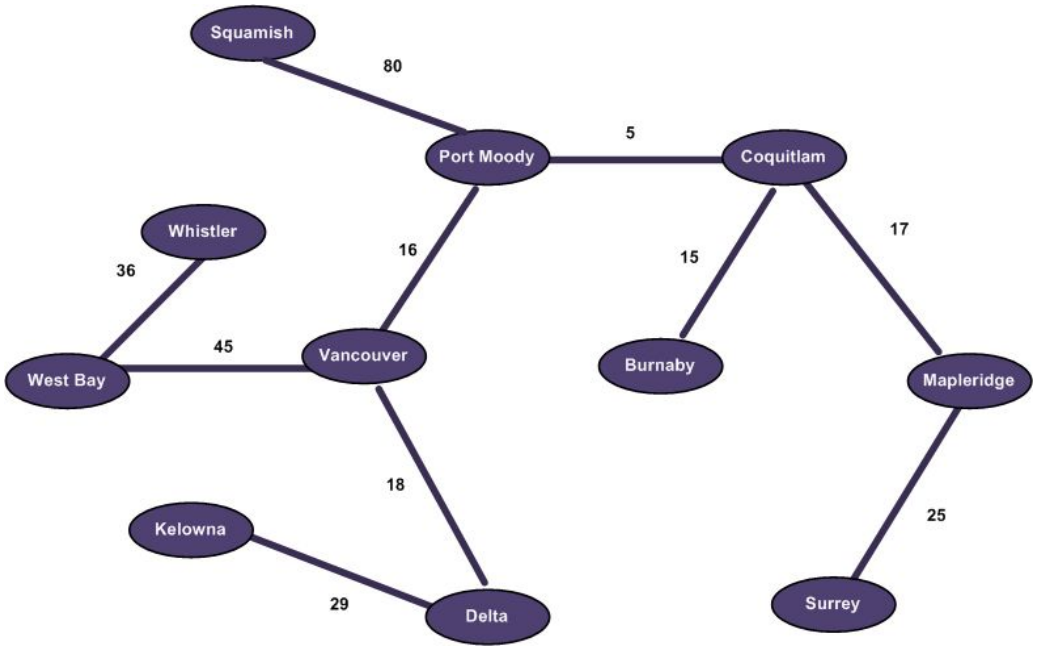
At the end port Moody to Squamish has the least weight, so we add Squamish as our last city of the visited list:

{Whistler, West Bay, Vancouver, Port Moody, Coquitlam, Burnaby, Maple ridge, Delta, Surrey,Kelowna, Squamish}

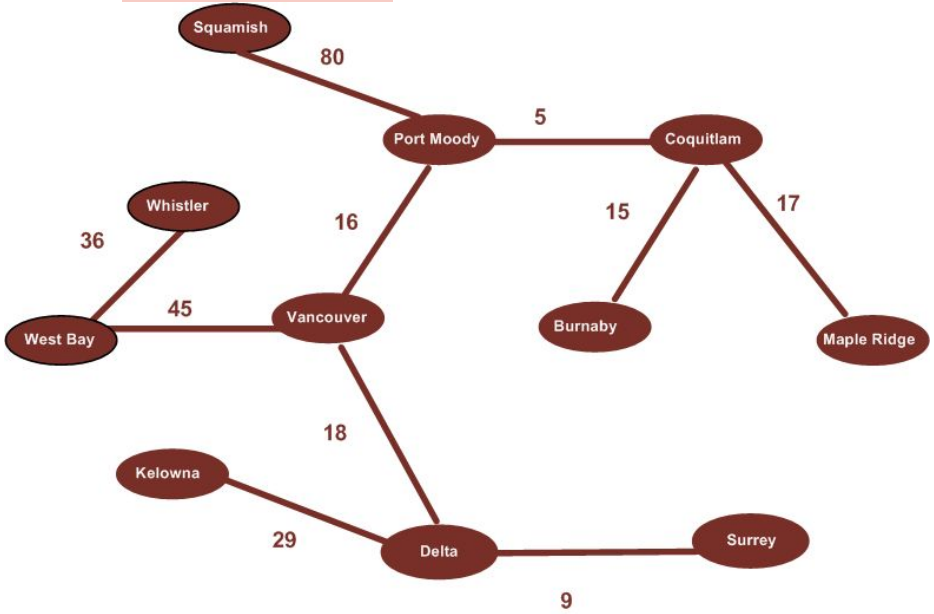
Eco-Friendly Travel Planner: Prim's Algorithm

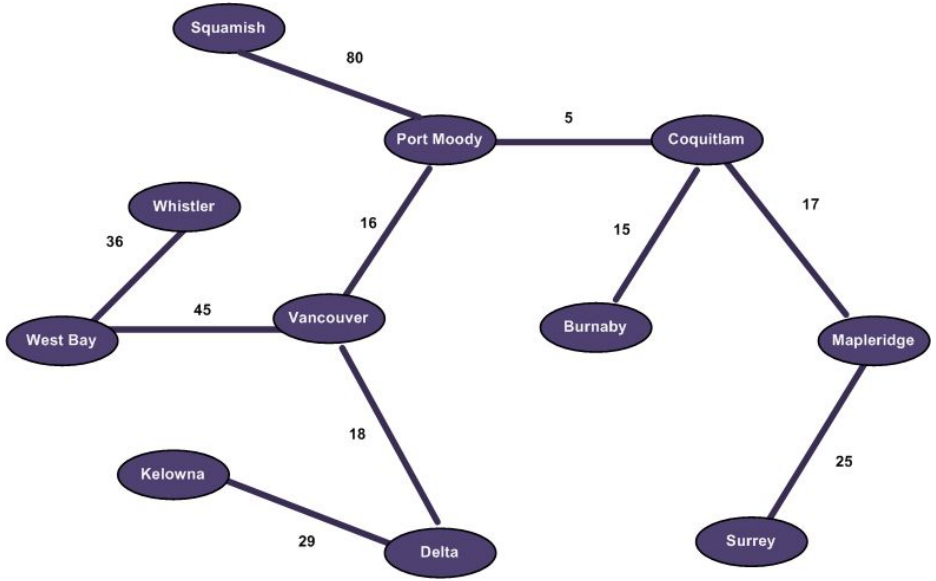


Eco-Friendly Travel Planner: Prim's Algorithm



Kruskal's & Prim's:





THANK YOU!

QUESTIONS?